

8051 Microcontroller: Interrupts & External I/O Interfacing

1 External I/O Interfacing

External I/O Interfacing Concept

External I/O devices in the 8051 microcontroller are interfaced using the **memory-mapped I/O** technique. In this approach:

- External devices are treated as **external memory locations**.
- Each device is assigned a unique address from the external memory address space.
- The CPU accesses these devices using standard memory read/write instructions (MOVX).

1.1 Memory-Mapped I/O Advantages

- **Unified Address Space:** I/O and memory share the same address bus, simplifying hardware design.
- **Standard Instructions:** No special I/O instructions are required; memory access instructions work for I/O.
- **Flexibility:** Any memory-addressing mode can be used to access I/O devices.

1.2 Trade-off: Address Space Consumption

Address Space Usage

Since external I/O devices are treated as memory locations, they **consume addresses** from the external memory space. This reduces the available address range for actual memory (RAM/ROM).

2 Typical System Memory Configuration

The 8051 can interface with external memory and peripherals. A typical system configuration includes:

Component	Size	Purpose
External RAM	16 KB	Data storage, stack, variables
External ROM	16 KB	Program code storage
8255 PPI	–	Programmable Peripheral Interface

8255 PPI Overview

The **Intel 8255 Programmable Peripheral Interface (PPI)** is a general-purpose I/O device that provides:

- **24 I/O lines** organized into three 8-bit ports (Port A, Port B, Port C).
- **3 operating modes:** Mode 0 (Basic I/O), Mode 1 (Strobed I/O), Mode 2 (Bidirectional).
- External interfacing for keyboards, displays, ADCs, DACs, etc.

When interfaced externally with 8051, the 8255 occupies addresses in the external memory space.

3 Interrupt System Overview

What is an Interrupt?

An **interrupt** is a signal that temporarily halts the normal program execution, allowing the CPU to respond to an event by executing a special routine called the **Interrupt Service Routine (ISR)**. After the ISR completes, the CPU resumes normal execution.

3.1 Why Use Interrupts?

- **Real-time Response:** Handle time-critical events immediately without polling.
- **Efficient CPU Utilization:** The CPU can perform other tasks instead of constantly checking for events.
- **Event-Driven Programming:** Code executes only when specific events occur.

3.2 8051 Interrupt Sources

The 8051 microcontroller has **5 interrupt sources**:

Priority	Interrupt Source	Vector Address	Flag
1 (Highest)	External Interrupt 0 (INT0)	0003H	IE0
2	Timer 0 Overflow	000BH	TF0
3	External Interrupt 1 (INT1)	0013H	IE1
4	Timer 1 Overflow	001BH	TF1
5 (Lowest)	Serial Port (RI/TI)	0023H	RI or TI

4 External Interrupts (INT0 and INT1)

External Interrupts

The 8051 provides **two external hardware interrupt inputs**:

- **INT0** (P3.2) – External Interrupt 0
- **INT1** (P3.3) – External Interrupt 1

These are used to detect external events from peripheral devices, sensors, or other hardware.

4.1 Trigger Modes

External interrupts can be configured to trigger in two different modes:

4.1.1 Edge-Triggered Mode

- Interrupt is triggered on a **high-to-low (falling edge)** transition.
- Configured by setting **IT0 = 1** (for INT0) or **IT1 = 1** (for INT1) in the TCON register.
- **Timing Requirements:**
 - Signal must remain **HIGH** for at least **1 machine cycle**.
 - Signal must remain **LOW** for at least **1 machine cycle**.
- The interrupt flags (IE0/IE1) are **automatically cleared** by hardware when the ISR is executed.

4.1.2 Level-Triggered Mode

- Interrupt is triggered when the input signal is at **LOW level**.
- Configured by setting **IT0 = 0** (for INT0) or **IT1 = 0** (for INT1).
- **Timing Requirement:** Signal must remain **LOW** for at least **2 machine cycles** to be recognized.
- The interrupt flags (IE0/IE1) are **controlled by the external source** itself and are **NOT automatically cleared**.

Level-Triggered Caution

In level-triggered mode, if the external signal remains LOW after returning from the ISR, the interrupt will be triggered again, causing repeated interrupts. The external source must remove the LOW signal before the ISR ends.

4.2 External Interrupt Processing Flow

Processing Sequence

1. External interrupt occurs (falling edge or low level detected).
2. Corresponding flag (**IE0** or **IE1**) is set in the TCON register.
3. CPU completes the current instruction.
4. CPU pushes the Program Counter (PC) onto the stack.
5. CPU jumps to the respective interrupt vector address:
 - **INT0** → **0003H**
 - **INT1** → **0013H**
6. ISR executes.
7. RETI instruction returns control and pops PC from stack.

5 Timer/Counter Interrupts

The 8051 has **two 16-bit timers**: Timer 0 and Timer 1. Each can generate an interrupt when it overflows.

5.1 Timer vs Counter Mode

Operating Modes

Timer Mode:

- Uses the **internal oscillator clock** as the counting source.
- Clock frequency for timer = $\frac{\text{Oscillator Frequency}}{12}$ (each machine cycle).
- Used for generating time delays and periodic events.

Counter Mode:

- Counts **external pulses** applied at T0 (P3.4) or T1 (P3.5) pins.
- Each high-to-low transition on the pin increments the counter.
- Used for counting events like shaft rotations, pulses, etc.

5.2 Timer Modes (TMOD Register)

The **TMOD register** configures the operating mode for both timers. Each timer can operate in 4 different modes:

Mode	Name	Bits	Description
0	13-bit Timer	M1=0, M0=0	5 bits of TL + 8 bits of TH (0000H – 1FFFH). Used for 8048 compatibility.
1	16-bit Timer	M1=0, M0=1	Full 16-bit operation (0000H – FFFFH). Most commonly used mode.
2	8-bit Auto-Reload	M1=1, M0=0	TL counts 00H – FFH. On overflow, TL is reloaded from TH automatically.
3	Split Timer	M1=1, M0=1	Timer 0 becomes two independent 8-bit timers. Timer 1 is halted in this mode.

5.3 Timer Overflow and Interrupt Generation

Overflow Mechanism

- Timers act as **up-counters**, incrementing from 0000H towards FFFFH.
- When the timer reaches its **maximum count** and overflows:
 - The corresponding overflow flag (**TF0** or **TF1**) in TCON is set.
 - If the interrupt is enabled, an interrupt is generated.
- The overflow flags are **automatically cleared by hardware** when the CPU vectors to the ISR.
- Interrupt Vector Addresses:
 - Timer 0 → **000BH**
 - Timer 1 → **001BH**

5.4 TCON Register (Timer Control)

Bit	Name	Function
7	TF1	Timer 1 overflow flag (set on overflow, cleared by hardware)
6	TR1	Timer 1 run control (1 = start, 0 = stop)
5	TF0	Timer 0 overflow flag (set on overflow, cleared by hardware)
4	TR0	Timer 0 run control (1 = start, 0 = stop)
3	IE1	External Interrupt 1 flag
2	IT1	Interrupt 1 type (1 = edge, 0 = level)
1	IE0	External Interrupt 0 flag
0	IT0	Interrupt 0 type (1 = edge, 0 = level)

6 Serial Port Interrupt

Serial Communication Interrupt

The 8051 has a built-in **full-duplex serial port** that can generate interrupts when:

- A byte has been **completely received** (RI flag is set).
- A byte has been **completely transmitted** (TI flag is set).

Both flags share the **same interrupt vector address: 0023H**.

6.1 Serial Interrupt Flags (SCON Register)

- **RI (Receive Interrupt)**: Set by hardware when a complete byte is received.
- **TI (Transmit Interrupt)**: Set by hardware when a byte is completely transmitted.

Manual Flag Clearing Required

Unlike timer and edge-triggered external interrupts, the **RI and TI flags are NOT automatically cleared** by hardware. The ISR must explicitly clear these flags using software:

```
CLR RI    ; Clear receive flag
CLR TI    ; Clear transmit flag
```

6.2 Serial ISR Logic

Since both RI and TI share the same vector, the ISR must determine which event occurred:

Serial Port ISR Structure

```
SERIAL_ISR:
    JB RI, RX_HANDLER    ; Jump if receive interrupt
    JB TI, TX_HANDLER    ; Jump if transmit interrupt
    RETI

RX_HANDLER:
    CLR RI                ; Clear receive flag
    MOV A, SBUF           ; Read received data
    ; ... process received data ...
    RETI

TX_HANDLER:
    CLR TI                ; Clear transmit flag
    ; ... load next byte or signal completion ...
    RETI
```

7 Interrupt Enable Register (IE)

The **IE (Interrupt Enable)** register controls which interrupts are enabled or disabled.

Bit	Name	Function
7	EA	Global Enable: 1 = Enable all interrupts, 0 = Disable all
6	–	Reserved
5	ET2	Timer 2 interrupt enable (8052 only)
4	ES	Serial port interrupt enable
3	ET1	Timer 1 interrupt enable
2	EX1	External Interrupt 1 enable
1	ET0	Timer 0 interrupt enable
0	EX0	External Interrupt 0 enable

Enabling Interrupts

To enable any interrupt, two conditions must be met:

1. **EA (Global Enable)** must be set to 1.
2. The **individual interrupt enable bit** must be set to 1.

Example: To enable Timer 0 interrupt:

```
SETB EA      ; Enable global interrupts
SETB ET0     ; Enable Timer 0 interrupt
```

8 Interrupt Priority Register (IP)

The 8051 provides a **two-level interrupt priority system**: High and Low. The **IP register** assigns priority levels to each interrupt source.

Bit	Name	Function
7	–	Reserved
6	–	Reserved
5	PT2	Timer 2 priority (8052 only)
4	PS	Serial port priority
3	PT1	Timer 1 priority
2	PX1	External Interrupt 1 priority
1	PT0	Timer 0 priority
0	PX0	External Interrupt 0 priority

8.1 Priority Rules

Priority Mechanism

- **Bit = 1:** Interrupt has **HIGH** priority.
- **Bit = 0:** Interrupt has **LOW** priority.
- A high-priority interrupt can **preempt** a low-priority ISR.
- A low-priority interrupt **cannot** interrupt a high-priority ISR.
- Interrupts of the **same priority level cannot interrupt each other**.
- If multiple interrupts of the same priority occur simultaneously, **default polling sequence** applies (INT0 → T0 → INT1 → T1 → Serial).

9 Software-Generated Interrupts

Single-Step and Software Interrupts

The 8051 allows generating **interrupts through software** by manually setting the appropriate flags:

- Setting TF0/TF1 can trigger a timer interrupt.
- Setting IE0/IE1 can trigger an external interrupt.
- This technique is useful for:
 - **Debugging:** Single-step execution through code.
 - **Testing:** Testing ISR code without external hardware.
 - **Prioritized Execution:** Transferring control to specific routines.

10 Summary: Key Concepts

Quick Reference Summary

- **5 Interrupt Sources:** INT0, Timer 0, INT1, Timer 1, Serial Port.
- **External Interrupts:** Configurable as edge or level triggered via IT0/IT1.
- **Timer Interrupts:** Generated on overflow; flags auto-cleared.
- **Serial Interrupt:** RI/TI flags must be cleared manually in ISR.
- **IE Register:** Controls enabling/disabling of interrupts (EA must be set).
- **IP Register:** Sets two-level priority (high/low).
- **TCON Register:** Contains timer control and external interrupt flags.
- **Memory-Mapped I/O:** External devices consume external memory addresses.

Understanding the 8051 interrupt structure is essential for developing responsive, real-time embedded systems applications.